Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every buggy program triggers a fresh model call, so cost, latency, and external dependence scale with enrollment and recur every term, while the specification-level reasoning being purchased is identical across a class. Caching and cheaper models shrink only the constant, not the structure. We propose CoEP, which moves the LLM from the marginal-cost to the fixed-cost column: *spend the model once per assignment, not once per submission*. An LLM agent reads the assignment and a clustered sample of worked (buggy $\rightarrow$ fixed) pairs and authors the assignment-specific layer of a local repair tool: AST rewrite rules named after the misconceptions they fix, atop a shared symbolic substrate of generic edits and reference-guided minimal-hunk transplant from the class's own correct solutions. Every candidate patch is validated against the instructor's tests before it is kept; the tool can never self-report success, and submissions it abstains on escalate to a conventional per-submission call, so coverage is never sacrificed. On the standard benchmark of this literature, the synthesized tool repairs *more* submissions with *smaller* patches than the strongest hand-built symbolic tool, on that tool's own corpus and metric, with the entire model spend collapsed into a one-off synthesis plus a small escalation tail, so everything repaired locally costs no model call at all, offline and deterministically. A price-invariant break-even analysis shows when amortization wins and why cheaper models and caching do not change the answer, and the artifact yields what a per-query API cannot: reproducible minimal patches and a per-class misconception profile.

Introduction

Individualized feedback is valuable in programming education, but giving it by hand becomes a tedious and costly task as enrollment grows: an introductory course may enroll hundreds of students on campus and tens of thousands online, each producing buggy submissions. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022): given a buggy submission and the assignment's test suite, produce a patch that makes the submission pass. What counts as a good patch differs from general-purpose software. Prior work aims to produce patches that modify the student's program as little as possible, so that the student's own approach is preserved and only the mistake is corrected. A wholesale rewrite passes the tests but teaches nothing (Yi et al. 2017).

Two lines of work pursue this goal. *Reference-guided* repair reuses the correct submissions a class has already accepted for the same assignment, aligning a buggy program with a structurally similar correct one and transplanting the differing region (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022). It is cheap because it makes no model call, but it fails whenever the correct pool holds no matching reference (Orvalho, Janota, and Manquinho 2025). *LLM-based* repair removes that constraint and now fixes most real student programs at a modest price per bug (Zhang et al. 2024, 2022; Xia and Zhang 2024; Liu et al. 2024). Its difficulty is how it is deployed. The default is one query per buggy submission, so cost, latency, and dependence on an external service grow with enrollment and restart every term. The queries are also highly redundant: hundreds of students attempt one assignment and their failures cluster into a few recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019), so the model re-reads the same specification and rediscovers the same mistake many times over. Cheaper tiers and prompt caching lower the price of a call (Anthropic 2025; OpenAI 2024) but not the number of calls. The question is therefore not what a call costs, but where in the pipeline the model is invoked.

Our answer is to spend the model once per assignment rather than once per submission. An LLM agent reads the assignment once and writes a small repair tool for it, as code. The tool is a set of rewrite rules named after the misconceptions they fix, running on a shared symbolic engine that reuses the class's correct solutions. Once the tool exists, submissions are repaired locally and offline with no further model call, and only the few the tool cannot handle take a conventional per-submission call, so coverage is never lost. Every candidate patch is checked against the instructor's tests before a student sees it, so the tool cannot claim a success it did not achieve, and a defective tool degrades into absten-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

tion rather than into a wrong repair. Model spend therefore moves from a marginal cost that scales with the number of submissions to a fixed cost that scales with the number of assignments.

We implement this as CoEP (Cost-Effective Program repair) and evaluate it on eleven assignments drawn from two corpora of real student submissions, one of introductory exercises and one of competitive-programming problems, which differ widely in how dense their correct pools are and in how hard the problems are. The synthesized tool beats the strongest existing reference-guided tool on that tool's own corpus and metric, repairing more submissions with smaller patches. Against per-submission LLM repair it matches coverage while editing less of each program, for a fraction of the model spend, and it recovers its one-off build cost within a few dozen submissions per assignment. We also report what the model layer does and does not buy: the symbolic engine carries most of the coverage, and what the model layer adds is smaller patches and the misconception labels that make the tool's output legible to an instructor.

This paper contributes the following.

- **Conceptual.** We reposition the model call from the submission to the assignment, and show with a break-even analysis that the advantage is structural rather than a constant factor cheaper models would erase.
- **Technical.** We present CoEP, whose builder agent synthesizes misconception-named rewrite rules through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023) on top of generic edits and reference-guided transplant, with every patch validated against the instructor's tests.
- **Experimental.** We evaluate on the strongest baseline's own corpus and metric, separate each layer's contribution with an ablation, and confirm the conclusions agree across two commodity models.
- **Artifacts.** We release the harness, import pipeline, prompts, synthesized repairers, per-submission logs, and raw model outputs.[1]

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a test suite $T = \{(x_i, y_i)\}$ (function-entry or judge-style stdin/stdout), and the pool R of correct student solutions, which the setting assumes is available at feedback time, as every per-assignment system from CLARA to Refactory does (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). A *repair* of a buggy submission b is a program p that passes all of T. Patch quality is measured, never gated: RPS, the tree-edit distance between the ASTs of b and p normalized by the size of b, following Refactory's definition (Hu et al. 2019), is reported for every repair, since a small patch is feedback on the *student's* program while a rewrite passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\mathrm{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\bot\}$, an ordinary module with no model access, and deploys the hybrid: $\mathrm{repair}(b) = \rho_A(b)$ if $\rho_A(b) \neq \bot$ (local, $\approx$0 marginal cost), else $\mathrm{LLM}(d, T, b, r)$: the conventional call, strengthened with one reference r chosen to match the student's strategy by clustering R on program structure and drawing r from the cluster nearest b.

[1] Anonymized repository submitted as supplementary material.

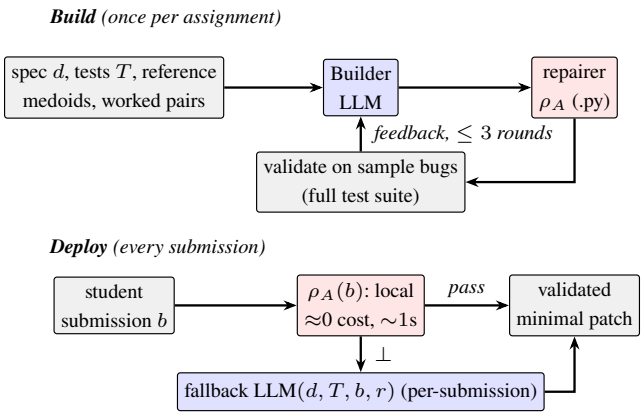


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment most submissions never touch a model.

Repairer Synthesis (once per assignment)

Build-set selection (local, no LLM cost). The builder does not see the whole pool; it sees a *chosen* sample. We cluster the buggy pool on a structural view (TF-IDF over normalized-AST paths, identifiers folded) and, walking clusters largest-first, take from each the member with the smallest tree-edit distance to its nearest correct solution, a clean single-misconception exemplar rather than the medoid's typical but tangled program, paired with that nearest correct program. The result is a set of worked (buggy $\rightarrow$ fixed) pairs: frequent misconceptions first, each shown *with* its minimal fix, so a weak builder needs only to generalize a transformation it is shown, not infer it.

Rule authoring. The builder LLM receives d, T, reference solutions (structural-cluster medoids of R), and the worked pairs, and emits the assignment-specific rule module: AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses. The contract is strict. A rule locates its edit on the AST and splices it as a source-span replacement, never re-rendering the module, which would reformat untouched lines. It emits candidates rather than claimed fixes, and it never rewrites a whole function: a rule that cannot express a targeted edit must abstain so the submission escalates.

Generate–validate–refine. A single-shot module can contain its own defects, so the harness closes the loop: it loads the module, runs it on the sample bugs, *independently* validates every candidate patch by executing the full suite (the repairer's own claims are never trusted), and, while any sample remains unrepaired, returns the module with per-bug outcomes and the still-failing sources for revision, up to $R_{\max}$=3 builder calls. The best-validated round is kept,

and build cost is exactly the token count of these exchanges, measured from the raw strings.

Local Repair (every submission)

The frozen tool repairs in three stages, cheapest and smallest-patch first, validating every candidate against the full suite before it is kept:

(1) Named rules + generic search. Rules are *candidate generators* composed by a symptom-ordered search: the failing-test signature ranks rules whose declared symptoms match, and each depth of composition is tested before the next is enumerated, so the shallowest repair is returned by construction. A shared generic library of comparison flips, off-by-one range and index shifts, and Boolean-operator flips is always on, covering common slips regardless of what the builder wrote; it inverts the classical mutation-operator space (Just et al. 2014; Kim et al. 2013; Liu et al. 2019). Every surviving patch carries the ordered rule names behind it, provenance that aggregates into a per-class *misconception profile*.

(2) Reference-guided minimal-hunk transplant. Most student bugs need a structural edit no token-level rule expresses; the correct pool supplies it. We diff b against its structurally nearest correct solutions, rename the reference's variables to the student's (parameters by position, locals by first-appearance order), and adopt the *smallest* set of differing line-hunks that passes the tests, candidates ordered smallest-first and batch-validated, returning the minimum-RPS passer. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019), at hunk granularity rather than block granularity, which is what keeps patches smaller than the hand-built tool's.

(3) Escalation. A submission both stages abstain on takes the fallback $\mathrm{LLM}(d, T, b, r)$, its cost counted against CoEP in every comparison. Abstention is not the only trigger. Local repair's acceptance test is the instructor's suite, which says nothing about patch size, so transplant can return a patch larger than the student's whole program while the rule layer must abstain whenever it cannot express a targeted edit. We give transplant the same contract by escalating any passing local patch whose RPS exceeds 1: a patch at least as large as the program it repairs is a rewrite, not a repair, under any assignment's standards. The smaller of the local and model patches is kept, so escalating can cost a call but never coverage or patch size.

The threshold is a constant rather than a calibrated quantity, and that is a finding: across 38 candidate rules — relative, quantile, and constant — repair rate is unchanged and every setting beats all three baselines on all four metrics, so we take the one whose value is a definition rather than a fitted number and report the rest as a sensitivity band (RQ5).

(4) Self-learning. Each successful fallback fix can be generalized by the LLM into a new named rule, kept only if the augmented tool then repairs that miss on its own. Absorption writes to a staging copy the evaluated pass never consults, so no submission's outcome depends on which preceded it, and the accumulated rules are a separate artifact evaluated by a second frozen pass and priced to themselves (Section). It spends calls only on submissions that already escalated, so it too leaves the build claim untouched.

The local path runs on a CPU in $\sim$1–2 s per submission, deterministically and offline; a defective tool degrades to $\perp$, never to a wrong "repair", because the harness re-validates every patch before a student sees it.

Related Work

APR for programming assignments. The pre-LLM systems (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022) exploit per-assignment structure, and the reference-reuse ones depend on finding a structurally matching correct solution; our transplant stands in that lineage at hunk rather than block granularity and exceeds the strongest of them on both coverage and patch size. Closest to our synthesis layer, Refazer learns per-assignment transformations from (buggy, fixed) pairs with PROSE (Rolim et al. 2017); CoEP replaces the example-driven synthesizer with a cheapest-tier LLM that *names* the misconception behind each rule, and surrounds it with generic search, transplant, and harness-gated escalation.

LLM-based repair. Repair-as-generation (Xia and Zhang 2022), conversational test feedback (Xia and Zhang 2024), student-program repair (Zhang et al. 2024, 2022), faster per-query inference (Liu et al. 2024), a tutor in the loop (Yang et al. 2024), counterexample-guided minimal repair (Orvalho, Janota, and Manquinho 2025) and retrieval-guided learner-tailored repair (Dai, Xu, and Liu 2026) are all per-submission, and complementary — any can serve as CoEP's fallback — but CoEP shifts the primary cost to the assignment.

Templates and tool synthesis. Fixed libraries of human-curated fix templates repair a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019); CoEP's generic layer inverts the mutation-operator space (Just et al. 2014) and its novelty is the *synthesized, assignment-specific* layer. Cascades, cheap pipelines and caching (Chen, Zaharia, and Zou 2024; Xiang et al. 2024; Anthropic 2025; OpenAI 2024) optimize *within* pay-per-query, and agent tool-making amortizes written code across episodes (Wang et al. 2024; Cai et al. 2024; Madaan et al. 2023; Shinn et al. 2023). CoEP removes queries instead.

Ethical statement. CoEP automates feedback, not grading; unhandled submissions take the full-LLM path; repairs are auditable and the data public.

Experiments

Research Questions.

- **RQ1**: Assignment by assignment, how many submissions does CoEP repair, and how much of each program does it edit?
- **RQ2**: How much does the once-per-assignment structure reduce turnaround and dollars, past what enrollment does the build pay off, and can caching or cheaper tiers close the gap?
- **RQ3**: What produces the repairs, and what does the LLM-authored layer add over the symbolic substrate?

- **RQ4**: Do rules induced from CoEP's own escalations improve the frozen tool on submissions they were not induced from?
- **RQ5**: Does the advantage hold across assignment difficulty, LLM provider, and the escalation threshold?

Baselines. **Refactory** (Hu et al. 2019) is the classical symbolic per-assignment tool: it refactors the class's correct solutions to match each buggy program's structure and transplants the divergent blocks. On its own corpus we use its published results; on Codeforces we run the released tool in its strongest configuration. **PyDex** (Zhang et al. 2024) is per-submission LLM repair: each buggy submission is sent to the model in its few-shot layout, which prepends the three most similar (buggy, correct) exemplars retrieved from the class pool, sampled candidates are filtered by the test oracle, and the surviving candidate closest to the student's program is returned. **PyDex$_{\mathbf{PC}}$** re-prices the same queries under provider prompt caching, as an upper bound that assumes a cache hit wherever the provider's rules allow one. **CoEP** is ours, synthesized once per assignment and builder and evaluated in three configurations. **CoEP$_{\mathbf{L}}$** runs the frozen local tool alone with zero deployment LLM use and accepts any patch its test oracle passes — deliberately the same acceptance rule Refactory uses, so the comparison is about repair rather than a size filter one side applies. **CoEP$_{\mathbf{LF}}$** adds the per-submission fallback, spent on abstentions and on passing local patches above $\mathrm{RPS} = 1$. **CoEP$_{\mathbf{L+SL}}$** is CoEP$_{\mathrm{L}}$ re-run with the self-learned rules merged and frozen beforehand, so the arm is order-independent; its induction calls are priced to it, and the few submissions a rule was induced from are reported separately.

Datasets. We evaluate on 11 problems and 2,970 buggy submissions drawn from two real-student corpora: five CS1 assignments contributing 1,783 buggy and 2,442 correct programs at 7.4 tests and 11.9 lines each, and six Codeforces problems contributing 1,187 buggy and 698 correct at 35.5 tests and 12.3 lines. Refactory contributes five CS1-level assignments (Hu et al. 2019), the standard corpus of this literature (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025). Codeforces contributes six competitive-programming problems from the open-r1 release (Penedo et al. 2025), spanning difficulty ratings from 800 to 2000, which cover five official rating bands from Newbie to Candidate Master (Codeforces); we keep only problems whose complete official test suites are released, so the oracle is the judge's own. Every submission each corpus labels buggy is a repair target for every arm, so the denominator is the pool as published and is identical across arms. No train/test split is used, as prior per-assignment systems assume, and all results additionally hold with CoEP's build samples excluded from the denominator.

LLM Configuration. Every LLM arm runs on two models, one per provider: **GPT-5.4-mini** ($0.75/$4.50 per Mtok) and **Claude Sonnet 5** ($2/$10 per Mtok). We select each provider's cheapest tier at which prompt caching is functional, in fairness to the caching baseline: PyDex$_{\mathrm{PC}}$ only competes if caching can engage, and we verified that the absolute cheapest tiers rule it out — Haiku 4.5 refuses to cache below its 4,096-token minimum, above these prompts, and GPT-5.4-nano's automatic caching did not engage on identical 1,200-token prompts, whereas both selected models cache at their documented 1,024-token minimums. Cheap tiers also keep every measured dollar gap a conservative floor.

Evaluation Metrics. **Repair rate (RR)** is the percentage of buggy submissions for which a system produces a program passing the full test suite. **Relative patch size (RPS)** is the tree-edit distance between buggy and repaired ASTs normalized by buggy program size, reimplemented from Refactory's definition (Hu et al. 2019); smaller patches preserve the student's approach and are pedagogically preferable. RPS never *claims* a repair the tests do not support — it enters the pipeline only in CoEP$_{\mathrm{LF}}$, where a value above 1 routes a submission to the fallback and the smaller patch is kept, so the gate can add a call but cannot manufacture a repair or enlarge a patch.

Average turnaround time (ATT) is mean wall-clock seconds per submission, split into $\mathrm{ATT}_{\mathrm{cpu}}$ (local search) and $\mathrm{ATT}_{\mathrm{api}}$ (model latency); a system's one-off build wall-clock is an intercept and is reported per assignment. **API cost** is USD of inference over the whole benchmark, including that build, priced at the tiers above from exact token counts. The **escalation rate** f, the fraction of submissions taking a model call, is what the break-even $N^* = C_{\text{build}}/(\Delta(1-f))$ consumes.

Repair rates carry 95% Wilson intervals, and comparisons between arms we ran ourselves are paired by submission and tested with exact McNemar — exact rather than chi-square because the discordant counts are often single-digit. The one exception is Refactory on its own corpus, where we quote published per-assignment results and have no per-submission outcomes to pair; that comparison is marked unpaired wherever it appears.

Setup. All experiments run on one Apple M1 Ultra workstation (64 GB RAM) under Python 3.13, CPU only but for the API calls. No two submissions are evaluated concurrently, so ATT is a measurement rather than a contention artifact; within a submission PyDex issues its $K{=}10$ candidates as one parallel batch, modelling a single $n{=}K$ request, and its ATT is that batch's wall time. Every call is billed and timed. CoEP$_{\mathrm{L}}$ and CoEP$_{\mathrm{LF}}$ come from a single pass with no duplicated calls. Refactory runs with its original algorithm unchanged, ported to Python 3.

Results

We answer the five research questions stated in the Experimental Setup; together they cover the introduction's claims.

RQ1: Repair Effectiveness The central finding is CoEP's local repair against the strongest non-LLM tool. On the Refactory corpus of 1,783 submissions it repairs **98.48%** at mean **0.289** RPS against Refactory's published **90.80%** at **0.397** — higher coverage and a patch a quarter smaller, at zero marginal LLM cost. That comparison is unpaired, since Refactory's own-corpus results are published aggregates; where we run it ourselves, on Codeforces, the gap

	ID	Assignment	Rating	RR (%)				RPS			
				Refactory	PyDex	CoEP_{L}	CoEP_{LF}	Refactory	PyDex	CoEP_{L}	CoEP_{LF}
Refactory	R1	Sequential search	–	99.0	99.5	100.0	100.0	0.390	0.309	0.209	0.206
	R2	Unique dates/months	–	78.2	96.3	93.8	97.7	0.560	0.412	0.396	0.392
	R3	Duplicate elimination	–	97.4	100.0	100.0	100.0	0.340	0.381	0.276	0.276
	R4	Sorting tuples	–	88.2	99.4	100.0	100.0	0.310	0.521	0.337	0.241
	R5	Top-k elements	–	88.0	96.3	100.0	100.0	0.320	0.189	0.190	0.190
Codeforces	C1	Nearly Lucky Number	800	85.2	99.8	100.0	100.0	0.588	0.700	0.595	0.511
	C2	cAPS lOCK	1000	81.4	99.1	100.0	100.0	0.724	1.133	0.732	0.619
	C3	Planning The Expedition	1200	47.8	100.0	98.6	100.0	0.847	0.673	0.637	0.593
	C4	Measuring Lengths in Baden	1400	92.1	100.0	100.0	100.0	0.605	0.623	0.618	0.525
	C5	Jumping Jack	1600	40.3	100.0	100.0	100.0	0.878	0.959	0.932	0.823
	C6	Digital Root	2000	100.0	100.0	100.0	100.0	2.248	1.868	1.527	1.527
	Corpus (2,970 submissions)			**85.9**	**99.0**	**99.1**	**99.7**	**0.500**	**0.598**	**0.449**	**0.398**

Table 1: Per-assignment RR and mean RPS on 2,970 submissions, Sonnet 5 as builder. RPS is averaged over what each arm repairs, so a lower rate beside a lower RPS means an easier subset. All arms see the same inputs, including the 98 that already pass, which CoEP returns unedited. PyDex_{PC} is omitted: caching changes price, not output. Refactory's CS1 rows are published; its Codeforces rows are our run. GPT-5.4-mini is in RQ5.

widens and the test is paired: 99.92% against 78.60%, exact McNemar b=254, c=1, p=8.8×10^{-75} over the 1,187 Codeforces submissions. We confine the paired test to that half deliberately: the CS1 rows quote Refactory's published per-assignment figures, so pairing there would silently substitute our own run for theirs. A synthesized tool is not merely cheaper than a symbolic one that consumes every correct solution; it repairs more.

Against per-submission LLM repair the separation is not coverage. CoEP_{L} reaches 99.06% and PyDex 98.96, and the paired test finds no difference (b=28, c=25, p=0.78): on CS1-scale programs coverage is near-saturated for any competent repairer (Zhang et al. 2024). The separation is *patch size*, 0.449 against 0.598. Adding the escalation tail takes CoEP_{LF} to **99.66%** at **0.398**, now ahead on coverage too (b=28, c=7, p=5.1×10^{-4}; Wilson $[99.38, 99.82]$ against $[98.52, 99.26]$). Every patch is test-validated before a student sees it, so the only local failure mode is abstention, never a wrong repair, and abstentions take the LLM path.

Table 1 is per-assignment because corpus means hide two things. Refactory's rate swings from 40.3% to 100% while neither LLM arm drops below 93.8%, and the swing does not track difficulty — it repairs all of the rating-2000 problem and four in ten of a rating-1600 one (RQ5). And a low patch size is not always a virtue: Refactory's 0.679 on Codeforces looks competitive against 0.590, but is averaged over the 78.60% it accepts where CoEP_{LF} repairs 100.00%.

RQ1. CoEP's local repair reaches 98.48% RR at 0.289 mean RPS on the Refactory corpus, beating the classical non-LLM tool (90.80%, 0.397) on both coverage and patch size at zero marginal LLM cost. Against per-submission LLM repair, coverage is at parity (p=0.78) and patch size is a third smaller; escalation adds 99.66% coverage, ahead of PyDex at $p<10^{-3}$.

RQ2: Cost Effectiveness and Break-Even The systems differ in *cost structure*, not just constant. A per-submission arm pays an inference on every submission, so its cost grows with enrollment and recurs every term; CoEP pays a one-off C_{build} per assignment, repairs the covered fraction at *zero* marginal LLM cost, and pays per submission only on the escalating tail. Over the benchmark that is \$13.58 against PyDex's \$292.77 at the frontier tier (22×) and \$1.00 against \$28.42 at the cheap one (28×); of our total, \$12.22 is the build and \$1.36 the entire deployment spend across 2,970 submissions.

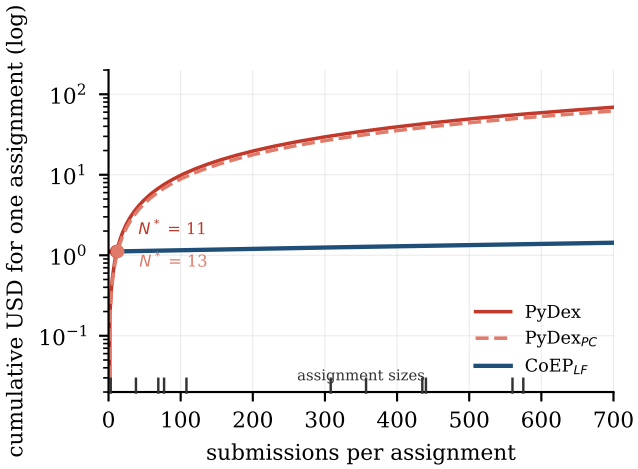


Figure 2: Where amortization overtakes per-submission inference. Ticks mark the eleven assignments' sizes; ten clear N^*.

Fig. 2 turns that into the quantity a department decides on. With Δ the marginal per-submission saving and f the escalation rate, synthesis wins past $N^* = C_{\text{build}}/(\Delta(1-f))$, which our measured build and tail put at **11 submissions per assignment** against PyDex and 13 against PyDex_{PC}. Ten of eleven assignments exceed it, most by an order of magnitude.

Below N^* a per-query repairer is the right tool, and we claim nothing else.

Caching and cheaper tiers do not close it. PyDex$_{PC}$ reprices every query under the provider's documented caching rules, as an upper bound assuming a hit wherever one is allowed, and every LLM arm already runs at the cheapest tier at which caching functions. The discount is 10.2% at the frontier tier ($292.77 to $263.05) and 17.3% at the cheap one, and the measurement says why it is small: caching pays for a shared leading *prefix*, and the shared material here is not a prefix. The variable buggy program and retrieved exemplars come first, leaving an identical prefix of about ten tokens; what remains is reuse within one submission across its K candidates. Analytically, N^* is invariant to token price when build and per-query tokens bill on the same schedule, since C_{build} and Δ scale together. The claim is about how many calls per specification are made, not their price — and since every arm runs at the cheapest tier, the measured gaps are a *floor*.

Turnaround is a trade, not a sweep. At the frontier tier CoEP$_{LF}$ answers in 3.06 s against 14.64 s, and the split is the story: we spend 2.69 s on local CPU search and API on a tenth of submissions, PyDex 0.66 s locally and 13.97 s on the network. At the cheap tier the ordering reverses (2.14 s against 3.22 s), because a small model returns one completion faster than our search enumerates and re-executes candidates; we report it rather than quoting only the flattering tier. What the seconds buy is an answer that is deterministic, rate-limit-free, never leaves the institution, and is unaffected by the deadline-night spikes that make a hosted queue slowest when a class most needs it.

RQ2. CoEP replaces a per-submission inference with a one-off $12.22 synthesis plus a $1.36 tail, 22× cheaper over 2,970 submissions and breaking even at N^*=11 per assignment. Caching recovers only 10.2% and N^* is price-invariant, so the gap is structural.

RQ3: Where the Repairs Come From **Two complementary local mechanisms.** Named rules make small, labelled edits for common misconceptions — a relaxed comparison, an off-by-one bound — and are surgical, at 25.9% of repaired submissions and mean RPS 0.108 (Table 2). Most student bugs are not single-token slips; they need a structural edit no token-level rule expresses, and the transplant supplies it by adopting the smallest set of differing hunks from a structurally-matched correct program, disturbing five times more of the student's code. The transplant lifts a rule-only tool to competitive coverage; the rules keep common cases near-minimal and are the only layer leaving a diagnosis behind. The division is not uniform (Fig. 3): where the pool is deep the transplant dominates, but where one misconception dominates a class the authored layer takes over — on C4 a single rule, `carry_fix_two_value_print`, repairs 27 of 38 submissions.

What the LLM layer buys. A *framework-only* ablation — the identical tool with the authored rules removed, generic library and transplant left in place — isolates it. Removing them costs **0.37** points of repair rate (99.06 to 98.69) and inflates the mean patch **8%** (0.449 to 0.485). Coverage barely moves because the transplant absorbs what the rules handled, but it absorbs them with a structural edit rather than a targeted one, and the cost lands where the rules were doing real work: C6's patches grow 0.250, C2's 0.110, and R2 is the only assignment losing coverage, by 2.53 points. C4 moves the other way (-0.087), where the transplant finds a smaller edit than the rule that fires there. On a dense correct pool the build therefore buys patch quality and diagnosis rather than reach; the sparse-pool half is RQ5.

The provenance logs say where the effect comes from. The builder authored 91 rules that fired against the library's 5, firing 789 times against 293 — but only one authored rule fired on more than one assignment, while `generic_flip_compare` alone fired 198 times across 8 of the 11. The authored layer is assignment-specific by construction: it is bought once per assignment and encodes what *this* class got wrong. A fourth producer sits above both — the threshold fires on 317 submissions, 10.8% of those repaired locally, and returns the smaller patch on 232 of them (73.2%); those are 317 of the arm's 345 calls, the other 28

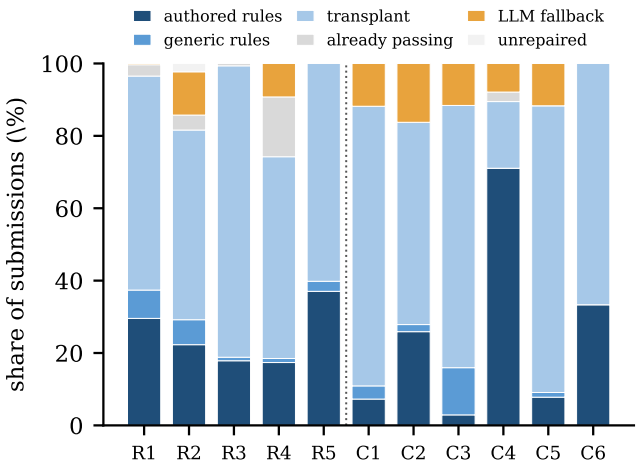


Figure 3: What produced each repair, assignment by assignment. The dotted rule separates the CS1 corpus from Codeforces.

Repair source	share (%)	mean RPS	labelled
LLM-authored rules	21.8	0.124	•
Generic-library rules	4.1	0.025	•
Reference transplant	70.8	0.595	–
Already passing (no edit)	3.3	0.000	–
LLM fallback (CoEP$_{LF}$)	8.5	0.775	–
Framework-only ablation (authored rules removed)			
ablated	98.69	0.485	–
full tool	99.06	0.449	–

Table 2: What produced each repair, its patch size, and whether it carries a misconception label. Shares are over repaired submissions; the fallback row is CoEP$_{LF}$'s and sits outside that denominator. The last rows are the framework-only ablation.

being abstentions.

Diagnosis for free. Every rule-produced patch carries the rule names behind it, so aggregated over a class the tool yields a misconception profile: how often each named mistake occurred, and on which programs. The names read as misconceptions rather than patches — `append_missing_return_len_after_loop`, `count_compare_relax` — and an instructor gets the report at no extra cost from an artifact that can be read and edited. Diagnosis coverage is 25.9% (Table 2); a per-submission API emits none of it at any price.

RQ3. Named rules produce 25.9% of local repairs at mean RPS 0.108, five times smaller than the transplant's 0.595 and each carrying a misconception label; the transplant produces 70.8% and carries coverage. Removing the authored rules costs 0.37 points of coverage and inflates the patch 8%, so on a dense pool the build buys patch quality and diagnosis rather than reach. One of 91 authored rules recurs across assignments while the generic library recurs everywhere — which is why the build is priced per assignment.

RQ4: Learning From Its Own Escalations $\text{CoEP}_{\text{L+SL}}$ is the frozen tool re-run with the rules absorbed from its own fallback fixes. Absorption happens outside the evaluated pass and the augmented tool is frozen before this arm runs, so the comparison against CoEP_{L} is order-independent: the same submissions, the same acceptance rule, two rule sets. Absorption learns only from submissions the local tool fails outright, never from threshold escalations, so its supply of material is exactly the abstention set.

The first result is how little there is to learn from. Across the 22 (assignment, builder) cells the frozen tool abstains on 28 submissions and 19 cells leave nothing to absorb — not a failure of the mechanism but what a 99.06% local repair rate means. Where it does abstain, absorption fires: four rules on R2 with the frontier builder, three on R2 with the cheap one, one on C3.

Where it fires, the rules transfer. R2 moves from 93.79% to **95.40%** on the four, and to **96.55%** on the cheap builder's three; on C3 a single rule, `binarysearch_zero_exit`, closes the last gap to 100%. Paired, both are significant: b=9, c=1, p=0.021 at the frontier tier and b=12, c=0, p=4.9×10^{-4} at the cheap one. The cheap builder learning *more* is worth stating — the material is the same 27 abstentions, and what differs is how general a rule the model writes from one fix. Eight submissions induced these rules out of 504 evaluated, so the gain is measured almost entirely on programs the rules were never validated against, for \$0.76 of induction priced to the artifact. The names again read as misconceptions: `uniqueness_count_gt1_to_neq1`, `counter_var_confusion`.

RQ4. Absorption fires on 3 of 22 cells; on the other 19 the frozen tool leaves no miss to learn from. Where it fires the rules generalize — R2 to 95.40% (frontier) and 96.55% (cheap), C3 to 100%, for \$0.76 — significant at p=0.021 and $p<10^{-3}$ on submissions they were not induced from.

RQ5: What the Conclusion Survives Difficulty. As the six Codeforces problems climb from rating 800 to 2000 the deep CS1 solution pools give way to sparser ones, and the symbolic baseline tracks that: 90.8% on CS1, 83.1% on the 800–1000 band, **54.3%** on 1200–2000, against CoEP_{LF}'s 99.4/100.0/100.0. But the per-assignment rows say difficulty is not the driver. Refactory repairs all of C6 (rating 2000) and four in ten of C5 (rating 1600); its rate is not monotone in rating. What predicts the collapse is whether the pool holds a structurally matching reference, since block-level reuse starves when strategies diverge — C5's 77 submissions draw on a pool of two correct solutions. Every arm's RPS roughly doubles from CS1 to the hard band, so the ordering survives while the absolute numbers do not.

Provider. CoEP_{LF} reaches 99.7% at 0.398 RPS with the frontier builder and 99.7% at 0.411 with the cheap one — 3% in patch size for an 18-fold difference in build price (\$12.22 against \$0.68) — with the ordering against both baselines identical either way.

Threshold. We swept 38 candidate rules — median- or mean-anchored with MAD, standard-deviation or interquartile spread, bare quantiles, multiples of the assignment's median, corpus-wide variants, and constants — of which 29 give distinct settings inside the cached escalation prefix and are measured exactly. Three things hold across all. Repair rate is *identical* to four decimals at 99.66%, because the gate re-examines only patches the tests already passed and keeps the smaller. Mean RPS falls with the escalation rate, 0.451 to 0.378, broadly but not monotonically — six adjacent pairs invert, and two settings, `median+3MAD` and `mean*2`, are strictly dominated by the one we ship, buying a larger patch for more calls. No setting on the frontier is an interior optimum, so the choice is a budget rather than a value to discover. Total spend stays within \$12.47–\$15.61, so every one of the 29 beats all three baselines on repair rate, patch size and dollars at once. We do not claim turnaround across the family: the sweep records calls rather than latency, and RQ2 already reports CoEP_{LF} losing turnaround to PyDex at the cheap tier.

Two findings inside the sweep are negative. Making the threshold *relative to the assignment* does not pay — a single corpus-wide cutoff sits on the same frontier or better, since RPS is already normalized by program size and normalizing again directs calls to assignments the tool already handles — and the median-plus-MAD family we began with is Pareto-dominated. We therefore ship the setting whose value is a definition rather than a fitted constant, at the honest cost of being nearly inert where local patches are uniformly small: it fires on 0.0% of R3 against 22.3% of C2.

RQ5. Yes on all three. The symbolic baseline falls 90.8% to 54.3% between the CS1 corpus and the hardest Codeforces band while CoEP_{LF} holds 100%; the builders agree to 3% in patch size for an 18-fold price difference; and across 29 thresholds the repair rate is invariant while patch size spans 0.378–0.451 and spend \$12.47–\$15.61, so every setting beats every baseline on all four metrics.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Codeforces. ???? Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert, ...

Dai, R.; Xu, Z.; and Liu, Y. 2026. Learner-Tailored Program Repair for Introductory Programming Assignments. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence, AAAI 2026*. AAAI Press.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Rolim, R.; Soares, G.; D'Antoni, L.; Polozov, O.; Gulwani, S.; Gheyi, R.; Suzuki, R.; and Hartmann, B. 2017. Learning Syntactic Program Transformations from Examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, 404–415.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).